

Reviewer Pack — Gate-Cone Poset Mobius Value Lower-Bounds ACC0 Size for MOD-...

Entry 3b280d479739 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-17 15:23:55 UTC

Gate-Cone Poset Mobius Value Lower-Bounds ACC0 Size for MOD-q

Entry ID: 3b280d479739 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-17 15:23:55 UTC

1. Conjecture

Field A (mathematical branch): Mobius functions on incidence algebras of finite posets (Rota 1964 combinatorial inversion; Stanley EC1 Ch.3) applied to the gate-dependency poset of an ACC^0 circuit; an algebraic-combinatorial invariant essentially absent from circuit complexity (arXiv search ‘Mobius function poset’ AND ‘ ACC^0 ’ returns no hits; <5 adjacent papers on poset invariants for circuits) **Field B** (complexity object): $\text{ACC}^0[m]$ circuit size for explicit symmetric functions (notably MOD_q for q coprime to m) in the Williams 2011 / Murray-Williams 2018 algorithmic-method regime, where structural circuit invariants underlie speedup-style lower bounds ($\text{NEXP} \not\subseteq \text{ACC}^0$)

Statement:

Let C be an $\text{ACC}^0[m]$ circuit on n inputs with s gates and depth d . Adjoin a formal minimum 0 below all gates and let $P(C)$ be the poset on $\{0\} \cup \text{Gates}(C)$ with $g \leq h$ iff $h = g$ or there is a directed wire path $g \rightarrow h$ (and $0 < g$ for every gate g). Define $M(C) := |\mu_{P(C)}(0, g_{\text{out}})|$ where μ is the Mobius function of $P(C)$ and g_{out} is the output gate, computed by $\mu(0,0)=1$ and $\mu(0,h) = -\sum_{0 \leq z < h} \mu(0,z)$. Conjecture (A, upper anchor): $M(C) \leq (s+1)^d$ for every $\text{ACC}^0[m]$ circuit. Conjecture (B, hardness anchor): for every $\text{ACC}^0[m]$ circuit C that correctly computes MOD_q on n inputs with $\gcd(q,m)=1$, one has $M(C) \geq 2^{\{c \cdot n\} \{1/(d+1)\}}$ for an absolute constant $c \geq 1/8$; a single counterexample circuit refutes (B).

Rationale (proposer’s reasoning):

Mobius inversion on the gate-dependency poset is a CIRCUIT invariant (not a truth-table property), so it dodges the natural-proofs trap that killed the Plunnecke variant in this approach; it is also non-ring (no polynomial extension makes the recursion $\mu(0,h) = -\sum \mu(0,z)$ meaningful over $\mathbb{F}_p[X]$), so it should not algebrize. Heuristically, Smolensky’s low-degree-polynomial obstruction for MOD_q in $\text{ACC}^0[m]$ forces long, non-factorisable dependency chains in any small circuit, which inflate alternating-sign Mobius cancellations on antichains of the gate poset; the $M(C)$ invariant captures this ‘inversion depth’ more sharply than gate count alone and is computable in $O(s^2)$ time, making it a viable Williams-style structural witness.

Taxonomy category: ACC_LB_via_WILLIAMS (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 32640d3f7431f4a2

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across all generated $ACC^0[2]$ instances ($n \in \{6, 8, 10, 12, 14, 16\}$, $d \in \{2, 3\}$, $s \in \{10, 20, 30, 40\}$, 30 seeds each), (A) every instance must satisfy $M(C) \leq (s+1)^d$; (B) among MOD_3-truth-table-verified circuits, $\geq 90\%$ per (n, d) cell must satisfy $M(C) \geq 2^{\lceil n \cdot \{1/(d+1)\}/8 \rceil}$. Any single (A)-violation or any MOD_3-correct circuit below the (B) bound falsifies.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.90	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Mobius function poset ACC^0 circuit lower bound - incidence algebra circuit complexity MOD_q symmetric function-gate dependency poset combinatorial invariant $ACC^0[m]$ depth

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import sys
import random
import math
import itertools
from collections import defaultdict

def generate_random_circuit(n, s, d, seed):
    random.seed(seed)
    gates = []
    for _ in range(s):
        if len(gates) < 2:
            gate_type = random.choice(['AND', 'OR', 'MOD_2'])
        else:
            gate_type = random.choice(['AND', 'OR', 'MOD_2', 'INPUT'])
        if gate_type == 'INPUT':
            inputs = []
        else:
```

```

        num_inputs = random.randint(2, min(3, len(gates)))
        inputs = random.sample(gates, num_inputs)
        gates.append((gate_type, inputs))
    return gates

def build_poset(circuit):
    poset = defaultdict(list)
    poset[0] = []
    for i, (gate_type, inputs) in enumerate(circuit):
        poset[i+1] = [0] + [j+1 for j, _ in enumerate(circuit) if any(j+1 in inp for inp in inputs)]
    return poset

def compute_mobius(poset, output_gate):
    mobius = {0: 1}
    for i in range(1, output_gate+1):
        if i not in poset:
            continue
        mobius[i] = -sum(mobius.get(j, 0) for j in poset[i] if j != i)
    return mobius

def is_mod3_circuit(circuit, n):
    for inputs in itertools.product([0, 1], repeat=n):
        output = evaluate_circuit(circuit, inputs)
        expected = sum(inputs) % 3
        if output != expected:
            return False
    return True

def evaluate_circuit(circuit, inputs):
    values = list(inputs)
    for gate_type, gate_inputs in circuit:
        if gate_type == 'AND':
            values.append(int(all(values[i] for i in gate_inputs)))
        elif gate_type == 'OR':
            values.append(int(any(values[i] for i in gate_inputs)))
        elif gate_type == 'MOD_2':
            values.append(sum(values[i] for i in gate_inputs) % 2)
        else:
            values.append(0)
    return values[-1]

def run_trial(seed):
    n_values = [6, 8, 10, 12, 14, 16]
    d_values = [2, 3]
    s_values = [10, 20, 30, 40]
    metric_values = []
    conjecture_holds = True
    counterexample = ""
    instances_tested = 0

    for n, d, s in itertools.product(n_values, d_values, s_values):
        circuit = generate_random_circuit(n, s, d, seed)
        poset = build_poset(circuit)
        output_gate = len(circuit)
        mobius = compute_mobius(poset, output_gate)
        m_c = abs(mobius.get(output_gate, 0))

```

```

    if m_c > (s + 1) ** d:
        conjecture_holds = False
        counterexample = f"M(C) = {m_c} > (s+1)^d = {(s+1)**d} for n={n}, d={d}, s={s}"
        break

    if is_mod3_circuit(circuit, n):
        threshold = 2 ** (n ** (1 / (d + 1)) / 8)
        if m_c < threshold:
            conjecture_holds = False
            counterexample = f"M(C) = {m_c} < threshold = {threshold} for MOD_3 circuit with n={n}, d={d}"
            break

    metric_values.append(m_c)
    instances_tested += 1

if not metric_values:
    return {
        "metric_name": "M(C)",
        "metric_value": 0.0,
        "instances_tested": 0,
        "conjecture_holds": False,
        "counterexample": "No valid instances generated"
    }

return {
    "metric_name": "M(C)",
    "metric_value": sum(metric_values) / len(metric_values),
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97]
    results = []
    metric_values = []
    conjecture_holds_counts = 0

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)
        metric_values.append(result["metric_value"])
        if result["conjecture_holds"]:
            conjecture_holds_counts += 1

    if not metric_values:
        print("RESULT: INCONCLUSIVE reason=no_metric_values")
        sys.exit(0)

    mean_metric = sum(metric_values) / len(metric_values)
    std_metric = math.sqrt(sum((x - mean_metric) ** 2 for x in metric_values) / len(metric_values))
    support_fraction = conjecture_holds_counts / len(seeds)

    if all(r["conjecture_holds"] for r in results):

```

```

    print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fr
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing
else:
    print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fbcfc841.py", line 123, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fbcfc841.py", line 78, in run_trial
    circuit = generate_random_circuit(n, s, d, seed)
              ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fbcfc841.py", line 27, in generate_rand
    num_inputs = random.randint(2, min(3, len(gates)))
              ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/random.py", line 336, in randint
    return self.randrange(a, b+1)
           ^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/random.py", line 319, in randrange
    raise ValueError(f"empty range in randrange({start}, {stop})")
ValueError: empty range in randrange(2, 1)

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a ValueError in generate_random_circuit (empty range in randrange(2,1)) before any instances were evaluated, so neither the support nor falsification condition could be assessed. | next: Fix the circuit generator to guarantee len(gates) ≥ 2 (or clamp the fan-in lower bound) before sampling num_inputs, then rerun the pre-registered sweep.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	104893
2	preregistration	claude_max	opus	0	0	7842
3	novelty	claude_max	opus	0	0	4772
4	novelty	claude_max	opus	0	0	5591
5	test_gen	mistral	codestral-latest	0	0	309080
6	test_gen	mistral	codestral-latest	0	0	310173
7	test_gen	mistral	codestral-latest	0	0	309637
8	test_gen	mistral	codestral-latest	0	0	317712
9	judge	claude_max	opus	0	0	4495

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 1374195 ms total latency. Provider mix: {'claude_max': 5, 'mistral': 4}

(full prompt+response transcripts available in `research/audit/3b280d479739.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/3b280d479739.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/3b280d479739.tar.gz` (if generated)